

# Esercitazione di Lab. di Sistemi Operativi

- *Fork & Exec* -

# Esercizio n° 1 (execl)

- Scrivere due file sorgenti in C: progA.c e progB.c.
- Ogni programma deve stampare un messaggio che identifichi il programma e mostri il proprio PID.
- ProgA lanci ProgB con una execl
- Compilare ed eseguire i due programmi separatamente per osservare la differenza dei PID.

# Soluzione Esercizio n° 1 (write)

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    printf("Sono il programma 2, con PID %d\n", getpid());
    return 0;
}
```

# Soluzione Esercizio n° 1 (write)

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    printf("Sono il programma 2, con PID %d\n", getpid());
    return 0;
}
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main(void) {
    pid_t pid;
    int status;

    printf("Sono il programma 1 con PID %d\n", getpid());
    execl("./prog2", "prog2", NULL);

    // se exec fallisce
    perror("execl");
    exit(1);
}
```

## Esercizio n° 2 (execl)

- Scrivere due file sorgenti in C: progA.c e progB.c.
- Ogni programma scriva un messaggio che identifichi il programma e mostri il proprio PID.
- Il progA.c crei un figlio che lanci progB.c attendendo la terminazione del figlio
- Compilare ed eseguire i due programmi separatamente per osservare la differenza dei PID.

# Soluzione Esercizio n° 2 (exec)

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main(void) {
    pid_t pid;
    int status;
    printf("Sono il programma 1 con PID %d\n", getpid());
    pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid == 0) {
        // processo figlio
        printf("Sto per eseguire prog2 con PID %d\n", getpid());
        execl("./prog2", "prog2", NULL);
        // se exec fallisce
        perror("execl");
        exit(1);
    } else {
        // processo padre
        printf("Ho creato un figlio con PID %d\n", pid);
        wait(&status);
        printf("Il figlio è terminato con status %d\n", WEXITSTATUS(status));
    }
    return 0;
}
```

## Esercizio n° 3 (execvp)

- Scrivere un programma che crea un processo figlio che esegue `ls -l`
- Utilizzare prima `execl` e poi `execvp`

# Soluzione Esercizio n° 3 (execlp)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid;
    int status;

    printf("[PADRE] PID = %d\n", getpid());

    pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid == 0) {
        // Processo figlio
        printf("[FIGLIO] Eseguo 'ls -l' con execlp()\n");
        execlp("ls", "ls", "-l", NULL);
        // Se execlp fallisce
        perror("execlp");
        exit(1);
    } else {
        // Processo padre
        wait(&status);
        printf("[PADRE] figlio terminato con codice %d\n", WEXITSTATUS(status));
    }
    return 0;
}
```



## Esercizio n° 4 (execvp)

- Scrivere un programma che prende in ingresso un comando e lo esegue
- Utilizzare la read() per leggere il comando
- ./leggiComando who
- ./leggiComando ls
- ./leggiComando wc

# Soluzione Esercizio n° 4 (execlp)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_CMD 128

int main(void) {
    char comando[MAX_CMD];
    ssize_t nread;
    pid_t pid;
    int status;

    printf("Inserisci un comando da eseguire (es. ls, date, who): ");
    fflush(stdout); // importante, perché printf è bufferizzato

    // Legge da stdin (file descriptor 0)
    nread = read(STDIN_FILENO, comando, sizeof(comando) - 1);
    if (nread < 0) {
        perror("read");
        exit(EXIT_FAILURE);
    }

    // Termina la stringa letta
    comando[nread] = '\0';
```

# Soluzione Esercizio n° 4 (execlp)

```
pid = fork();
if (pid < 0) { perror("fork"); exit(EXIT_FAILURE);}

if (pid == 0) {
    // Processo figlio
    printf("[FIGLIO] Eseguo '%s' tramite execlp()\n", comando);
    execlp(comando, comando, (char *)NULL);

    // Se execlp fallisce
    perror("execlp");
    _exit(127);
}

// Processo padre
if (waitpid(pid, &status, 0) < 0) {
    perror("waitpid");
    exit(EXIT_FAILURE);
}

if (WIFEXITED(status)) {
    printf("[PADRE] Figlio terminato con codice %d\n", WEXITSTATUS(status));
} else if (WIFSIGNALED(status)) {
    printf("[PADRE] Figlio terminato da segnale %d\n", WTERMSIG(status));
}

return 0;
}
```

## Esercizio n° 5 (signal)

- Scrivere un programma che si stoppa dopo 3 Ctnl-C
- Occorre una variabile globale contatore e un handler

# Soluzione Esercizio n° 5 (signal)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

int counter = 0;    // contatore globale

void handler(int signo) {
    counter++;
    printf("\n[handler] Ricevuto SIGINT numero %d\n", counter);

    if (counter >= 3) {
        printf("[handler] Ho ricevuto 3 SIGINT, termino.\n");
        exit(0);
    }
}

int main(void) {
    if (signal(SIGINT, handler) == SIG_ERR) {
        perror("signal"); exit(1); }
    printf("PID = %d\n", getpid());
    printf("Premi Ctrl-C (3 volte per uscire)\n");

    while (1) { printf("Lavoro...\n"); sleep(2); }

    return 0;
}
```

## Esercizio n° 6 (signal)

- Scrivere un programma C “aspetta.c”, che scrive un messaggio su standard output ogni volta che riceve i segnali SIGINT o SIGUSR1.
- Il programma non deve mai terminare spontaneamente.
- Usando il comando kill da una nuova shell, provare ad inviargli quei due segnali, ed infine terminarlo.

# Soluzione Esercizio n° 6 (signal)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void on_sig(int s) {
    if (s == SIGINT)    printf("[aspetta] Ricevuto SIGINT\n");
    if (s == SIGUSR1)   printf("[aspetta] Ricevuto SIGUSR1\n");
    // torna al punto interrotto
}

int main(void) {
    if (signal(SIGINT, on_sig) == SIG_ERR) { perror("signal"); exit(1); }
    if (signal(SIGUSR1, on_sig) == SIG_ERR) { perror("signal"); exit(1); }

    printf("PID=%d - invia SIGINT o SIGUSR1 con 'kill -SIGINT %d' / 'kill -\n",
           getpid(), getpid(), getpid());

    for (;;) pause();    // attende segnali indefinitamente
}
```

# Esercizio n° 7 (signal)

- Lanciare ./aspetta
- Sospendere con Cntrl+Z
- Lanciare ancora ./aspetta
- Sospendere con Cntrl+Z
- Lanciare jobs
- Mettere in bg (background) il primo: bg %1
- Mettere in fg (foreground) il secondo: fg %2